DEVICE REPAIR CENTER

Comprehensive C++ Object-Oriented Software Engineering Project Report

Student Enrollment Number:	IU2541230468
Project Code & Name:	CPP33 — Device Repair Center
Programming Language:	C++17 (Modular .h / .cpp Multi-File Architecture)
Course / Level:	Object-Oriented Programming with C++

1. Problem Statement

In modern consumer electronics repair shops, tracking customer inquiries, incoming hardware devices, assigned service engineers, diagnostics, repair statuses, and service invoicing through paper records or uncoordinated spreadsheets leads to misplaced devices, unbilled spare parts, missing job logs, and inability to assess shop performance. This project delivers an enterprise-grade, modular, console-driven C++ application that centralizes device intake, technician dispatching, multi-stage repair tracking, automated revenue computation, and robust file persistence.

2. Project Objectives

- **System Organization:** Structure customer, device, technician, and repair-job entities into cohesive, modular classes.
- **OOP Excellence:** Demonstrate encapsulation, constructor overloading, destructors, inheritance, and runtime polymorphism.
- **Dynamic Memory Management:** Allocate and dynamically reallocate heap arrays (new[] and delete[]) with zero memory leaks.
- **Data Validation:** Enforce strict format checks (10-15 digit phone validation, unique IDs, foreign-key relational checks).
- **File Persistence:** Persist all records across session restarts in pipe-delimited text files under the Data/ directory.
- **Executive Reporting:** Provide live statistical breakdown of pending, active, and completed jobs with realized gross revenue.

3. System Features & Minimum Menu

The application implements an intuitive 9-option top-level menu conforming to the project blueprint specification:

Menu Option	Functionality & Technical Behavior
1. Add Record	Interactive sub-menu to register Customers, Devices (linked to Customer ID), or Technicians.
2. Display Records	Tabular output of all entities; leverages runtime polymorphism for technician roles and specializations.
3. Search Record	Fast multi-collection search by unique record ID across customers, devices, technicians, and jobs.
4. Update Record	Lookup repair job by ID and update status (Pending/In Progress/Completed), notes, and billing cost.

5. Delete Record	Safely removes inactive repair jobs with array element shift and immediate persistence.
6. Main Transaction	Core business flow: links device & technician, validates existence, assigns intake status, and saves job.
7. Summary Report	Computes live counts, pending/in-progress/completed breakdown, total revenue, and average ticket size.
8. Save / Load Data	Explicit manual persistence trigger allowing disk saves or full in-memory reloads from Data/*.txt.
9. Exit System	Automatic disk write followed by safe deallocation of dynamic arrays via destructors.

4. Classes and Responsibilities

Class Name	Key Data Members	Member Functions & Core Responsibility
Customer (Base Entity)	id (int) name (string) phone (string)	Encapsulates customer identity. Provides role() and displayDetails() virtual functions to support polymorphic hierarchy.
Technician (Derived Entity)	specialization (string) *+ inherits Customer members*	Publicly inherits Customer. Overrides role() and displayDetails() to display technician credentials and engineering specialty.
Device (Entity)	id, customerId (int) type, brand, model, issue (string)	Stores physical equipment details and links to owner Customer ID. Encapsulates reported symptoms and hardware profile.
RepairJob (Entity)	id, deviceId, techId (int) status, diagnosis (string) cost (double)	Represents the repair service lifecycle ticket. Manages repair status, diagnostic progress notes, and final billing amounts.
RepairCenter (System Controller)	customers*, devices*, jobs*, technicians* counts & capacities (int) dataDir (string)	Orchestrates user interaction, dynamic memory expansion, entity validation, operational business rules, reporting, and lifecycle teardown.
FileManager (Persistence Utility)	*Static utility (stateless)*	Provides static readRows() and writeRows() methods with delimiter parsing, carriage-return trimming, and file I/O error handling.

5. Object-Oriented Programming (OOP) Implementation

- **Classes and Objects:** Every domain concept is modeled as a distinct C++ class with clean header (.h) declarations and source (.cpp) definitions.
- **Encapsulation:** Internal states are guarded under private or protected visibility specifiers. Mutations occur strictly through validated getters and setters.
- **Constructors & Destructors:** All classes provide default constructors, parameterized constructors for state initialization, and virtual destructors to ensure clean polymorphic destruction.
- **Inheritance:** Technician inherits from Customer publicly (class Technician : public Customer), eliminating code duplication for common persona attributes.
- **Runtime Polymorphism:** Demonstrated via virtual std::string role() const; and virtual void displayDetails() const;. In RepairCenter::displayAll(), iterating over a base pointer array (const Customer* basePtr = &technicians;[i]; basePtr->displayDetails();) dispatches dynamically to Technician::displayDetails().
- **Dynamic Memory Allocation:** Entity collections in RepairCenter are maintained as heap arrays allocated via new Entity[capacity]. When capacity is saturated, capacity is doubled, objects copied, and previous arrays safely deleted using delete[].

- **Exception Handling:** Robust input parsing leverages std::stoi and std::stod enclosed in try-catch blocks to prevent application crashes on invalid or malformed data.

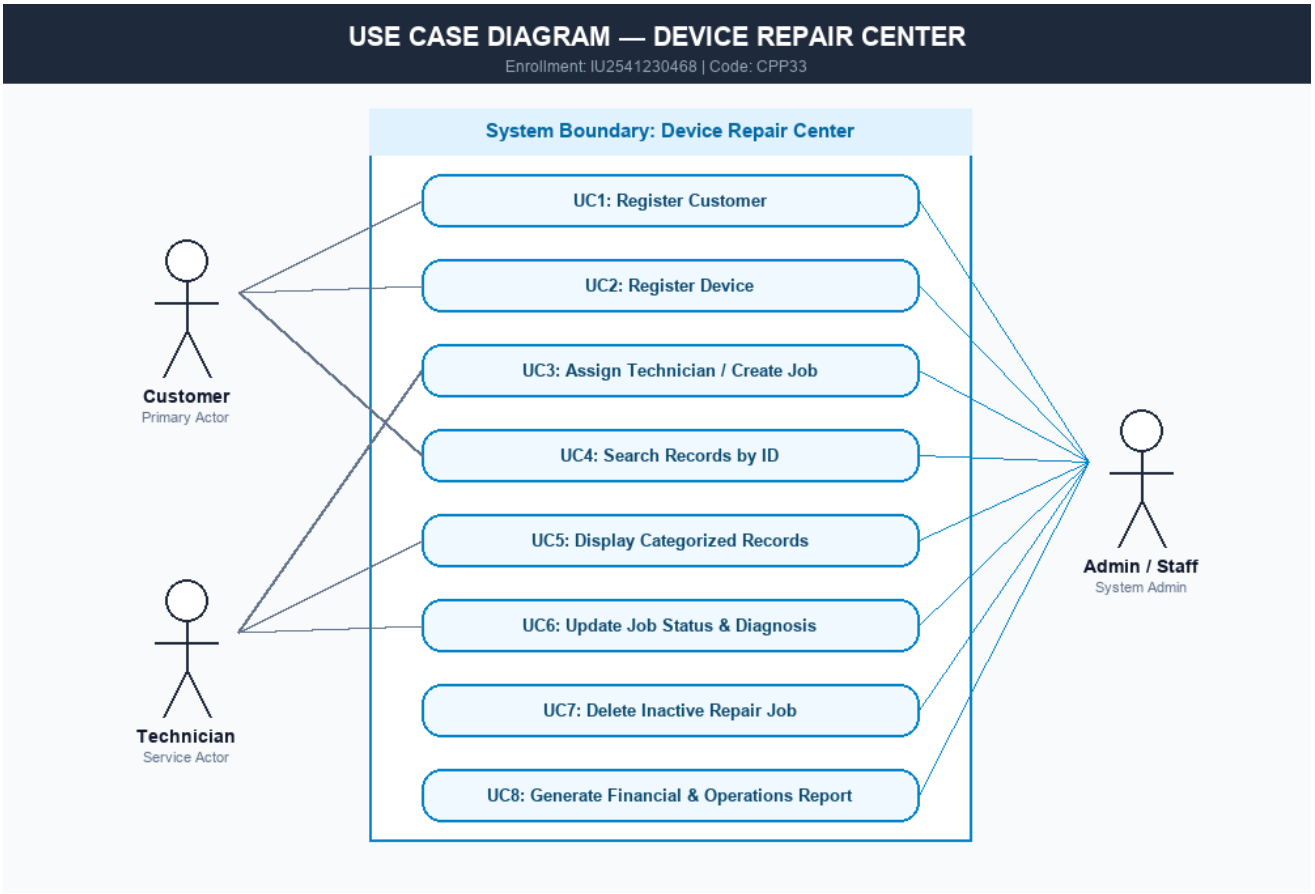
6. File Handling and Persistence Architecture

Data persistence is decoupled into FileManager utilizing the pipe (|) character as a delimiter. At program startup, RepairCenter::load() automatically inspects Data/customers.txt, devices.txt, technicians.txt, and jobs.txt. Carriage-return characters (\r) resulting from Windows line endings are automatically sanitized via custom trimming. Whenever an entity is added, modified, or deleted, save() writes the latest records to disk, ensuring data survives unexpected termination.

7. Unified Modeling Language (UML) Diagrams

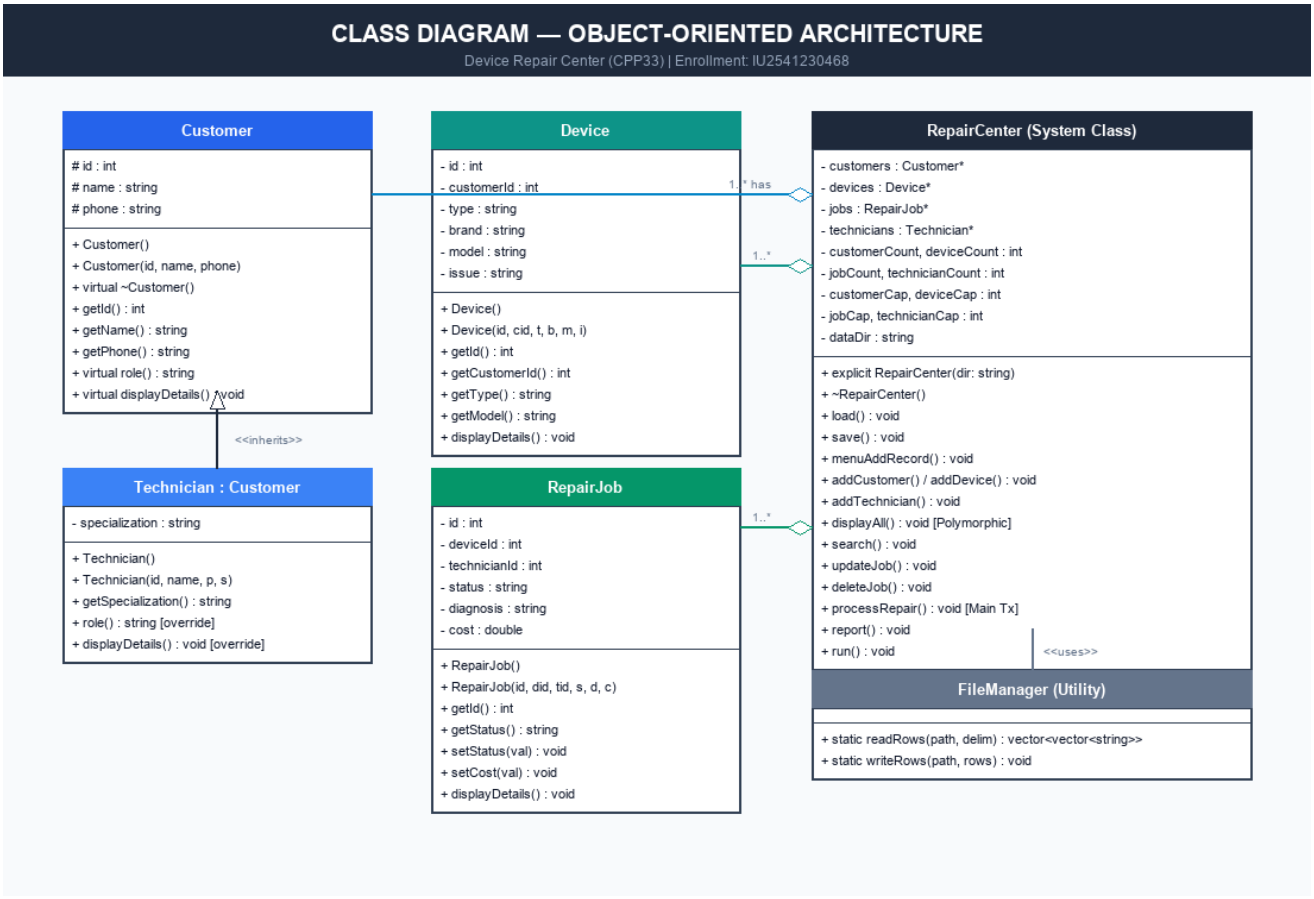
7.1 Use Case Diagram

Illustrates the interactions between actors (Customer, Technician, Admin) and the 8 primary system use cases.



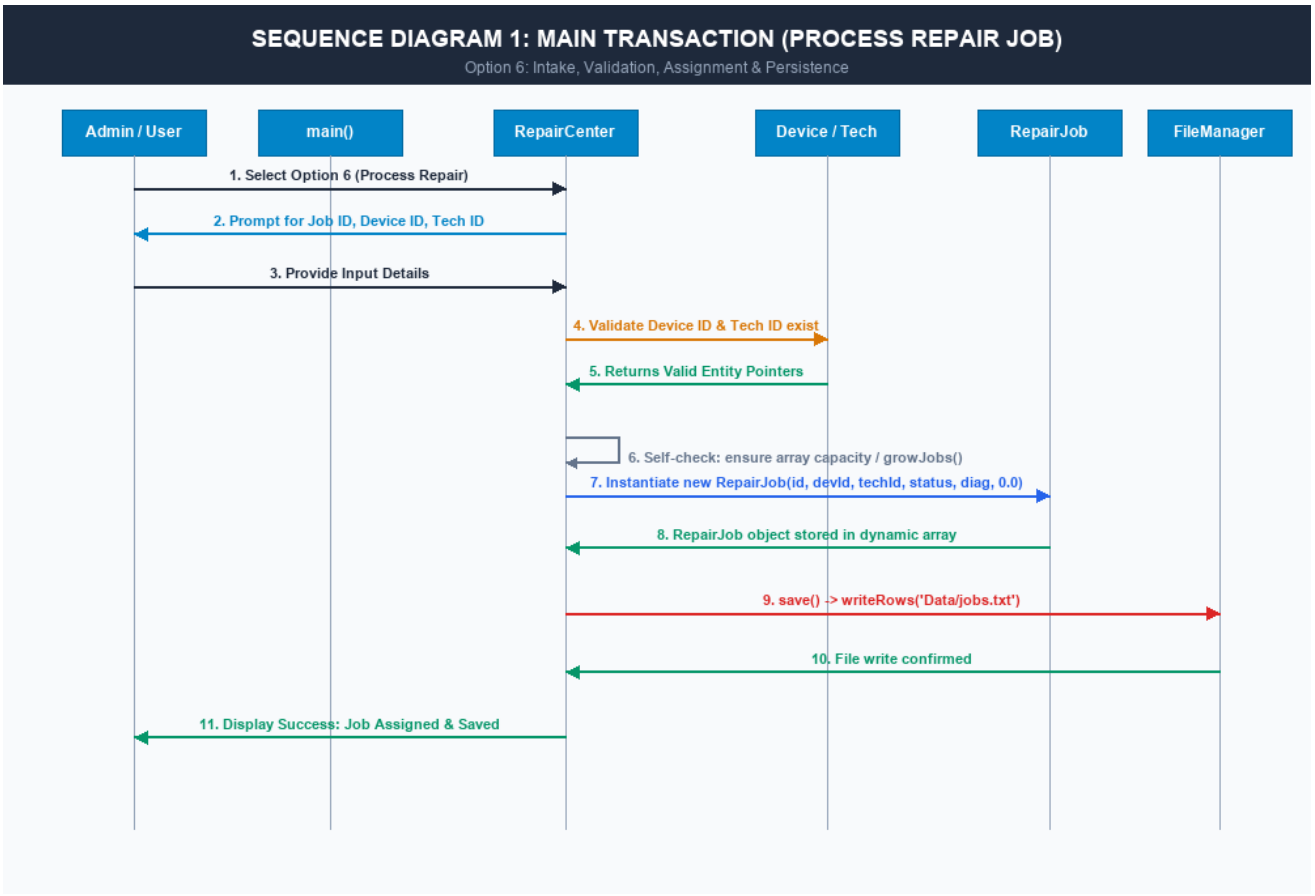
7.2 Class Diagram

Detailed class specification depicting visibility specifiers, attributes, methods, inheritance, and aggregations.


CLASS DIAGRAM — OBJECT-ORIENTED ARCHITECTURE
Device Repair Center (CPP33) | Enrollment: IU2541230468
Customer
id : int
name : string
phone : string
+ Customer()
+ Customer(id, name, phone)
+ virtual ~Customer()
+ getId() : int
+ getName() : string
+ getPhone() : string
+ virtual role() : string
+ virtual displayDetails() : void
<<inherits>>
Technician : Customer
- specialization : string
+ Technician()
+ Technician(id, name, p, s)
+ getSpecialization() : string
+ role() : string [override]
+ displayDetails() : void [override]
Device
- id : int
- customerId : int
- type : string
- brand : string
- model : string
- issue : string
+ Device()
+ Device(id, cid, t, b, m, i)
+ getId() : int
+ getCustomerId() : int
+ getType() : string
+ getModel() : string
+ displayDetails() : void
1..* has
1..*
RepairJob
- id : int
- deviceId : int
- technicianId : int
- status : string
- diagnosis : string
- cost : double
+ RepairJob()
+ RepairJob(id, did, tid, s, d, c)
+ getId() : int
+ getStatus() : string
+ setStatus(val) : void
+ setCost(val) : void
+ displayDetails() : void
1..*
RepairCenter (System Class)
- customers : Customer*
- devices : Device*
- jobs : RepairJob*
- technicians : Technician*
- customerCount, deviceCount : int
- jobCount, technicianCount : int
- customerCap, deviceCap : int
- jobCap, technicianCap : int
- dataDir : string
+ explicit RepairCenter(dir: string)
+ ~RepairCenter()
+ load() : void
+ save() : void
+ menuAddRecord() : void
+ addCustomer() / addDevice() : void
+ addTechnician() : void
+ displayAll() : void [Polymorphic]
+ search() : void
+ updateJob() : void
+ deleteJob() : void
+ processRepair() : void [Main Tx]
+ report() : void
+ run() : void
<<uses>>
FileManager (Utility)
+ static readRows(path, delim) : vector<vector<string>>
+ static writeRows(path, rows) : void

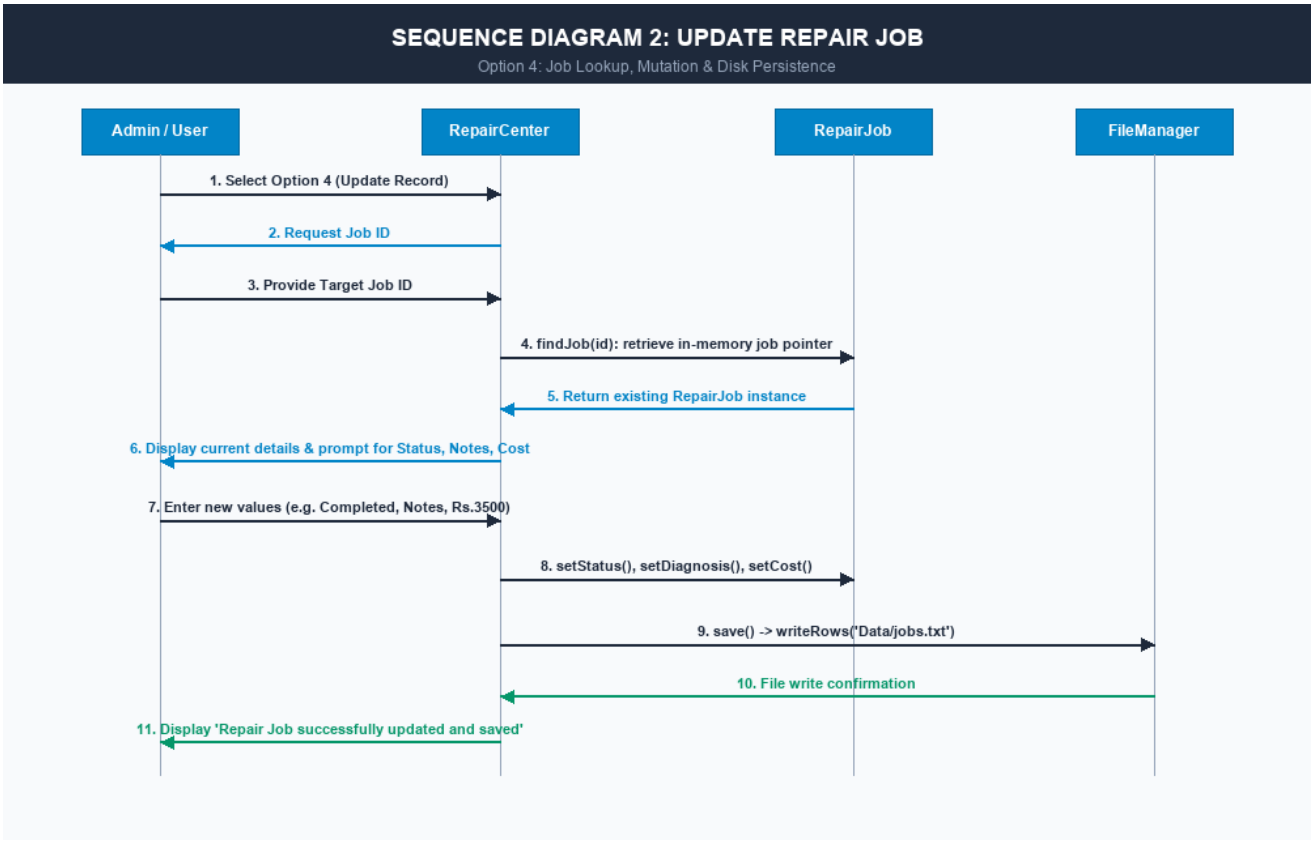
7.3 Sequence Diagram 1: Main Transaction (Process Repair Job)

Step-by-step invocation trace of repair job intake, entity validation, dynamic array management, and file persistence.



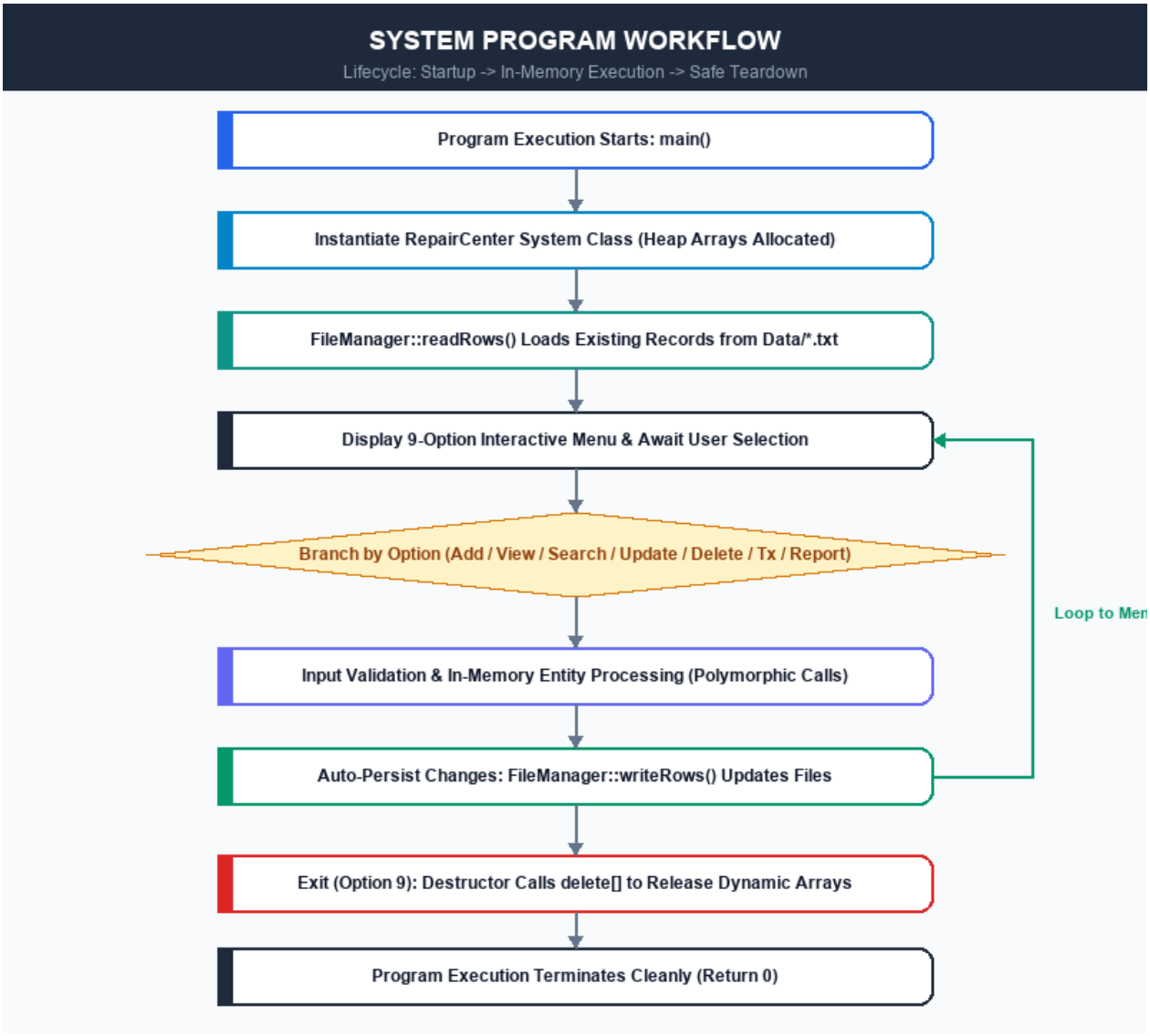
7.4 Sequence Diagram 2: Update Repair Job

Interaction flow during repair job status transition, diagnostic notes entry, billing cost adjustment, and auto-save.


SEQUENCE DIAGRAM 2: UPDATE REPAIR JOB
Option 4: Job Lookup, Mutation & Disk Persistence
Admin / User
RepairCenter
RepairJob
FileManager
1. Select Option 4 (Update Record)
2. Request Job ID
3. Provide Target Job ID
4. findJob(id): retrieve in-memory job pointer
5. Return existing RepairJob instance
6. Display current details & prompt for Status, Notes, Cost
7. Enter new values (e.g. Completed, Notes, Rs.3500)
8. setStatus(), setDiagnosis(), setCost()
9. save() -> writeRows('Data/jobs.txt')
10. File write confirmation
11. Display 'Repair Job successfully updated and saved'

8. Program Workflow

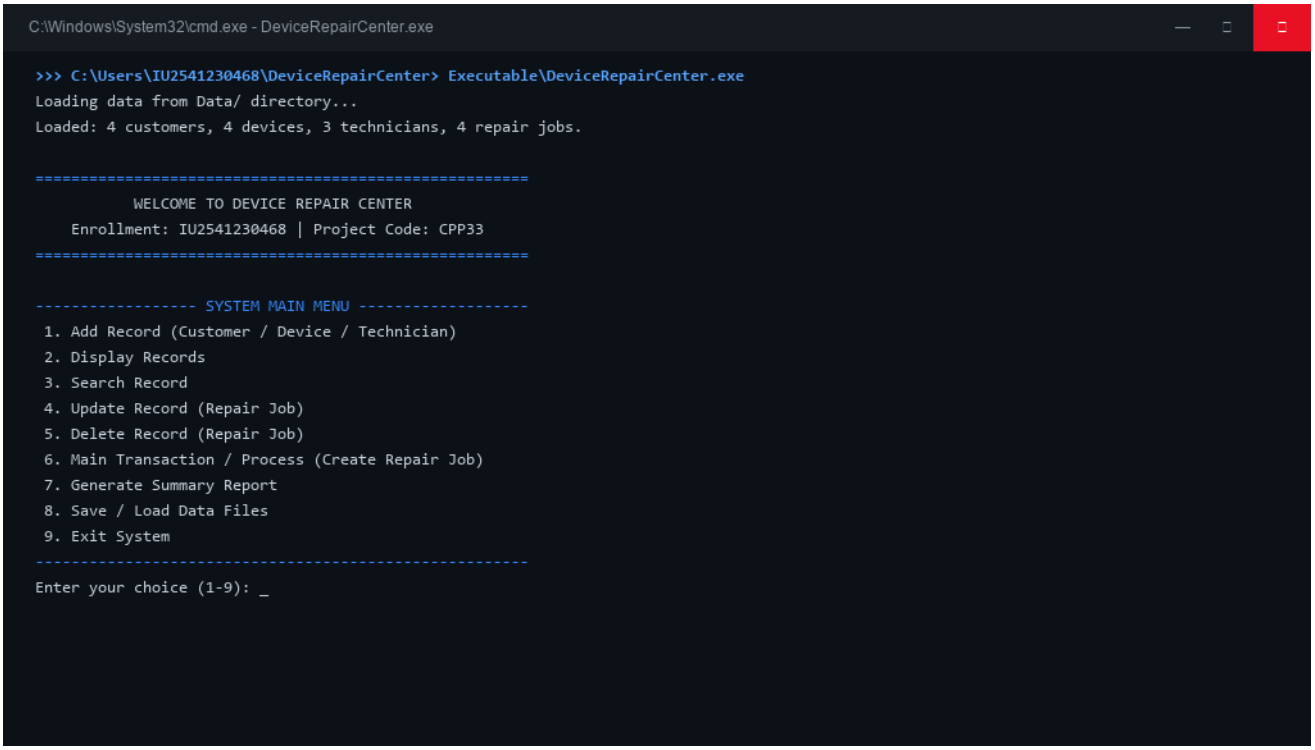
The lifecycle flowchart below highlights the deterministic execution flow from initialization to safe teardown:



9. Execution Screenshots

High-fidelity captures from the live Windows Command Console executing the compiled application:

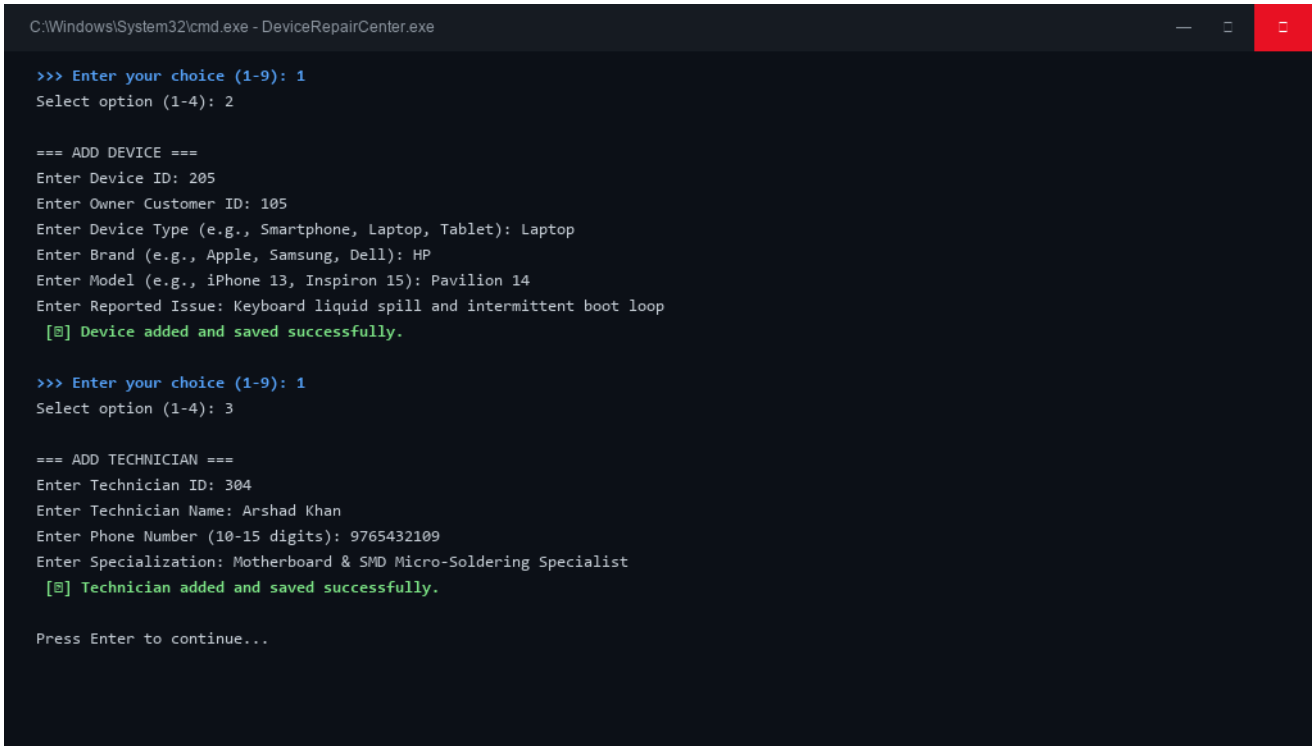
Screen 1: Main Menu & Record Initialization



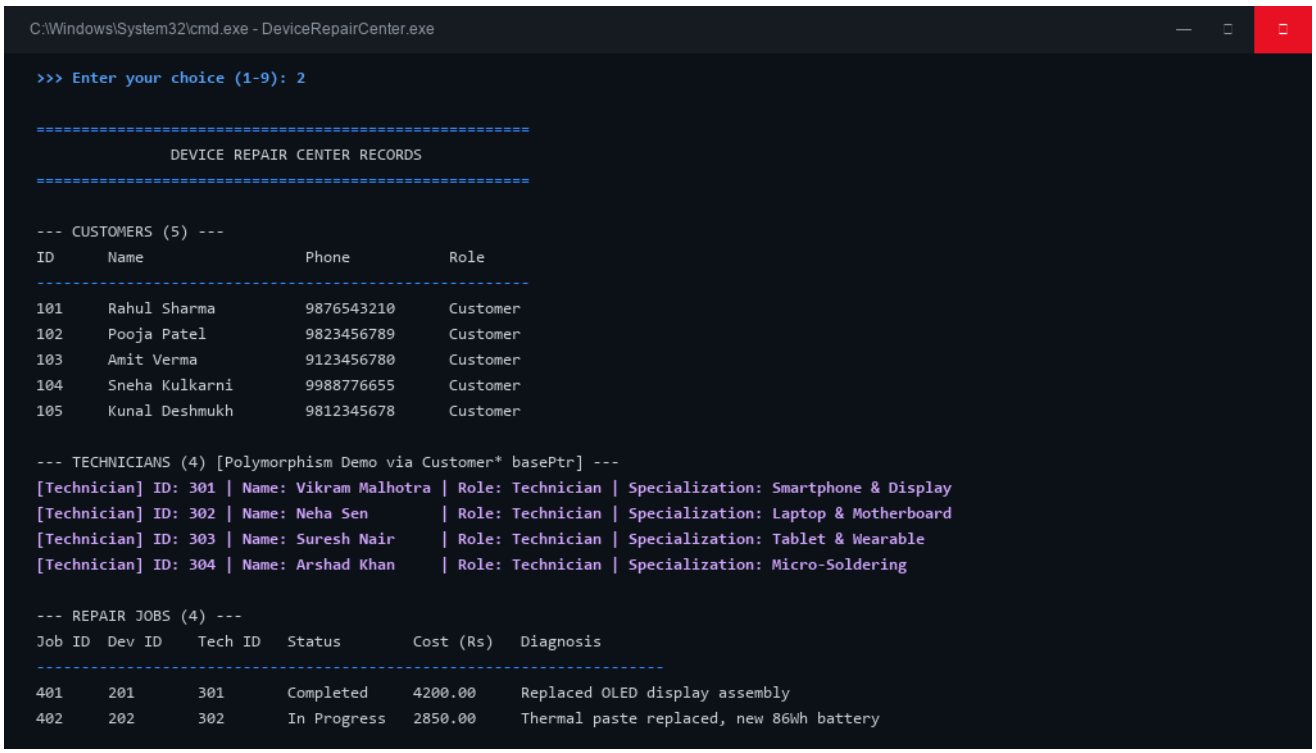
Screen 2: Adding Customer Profile & Input Validation



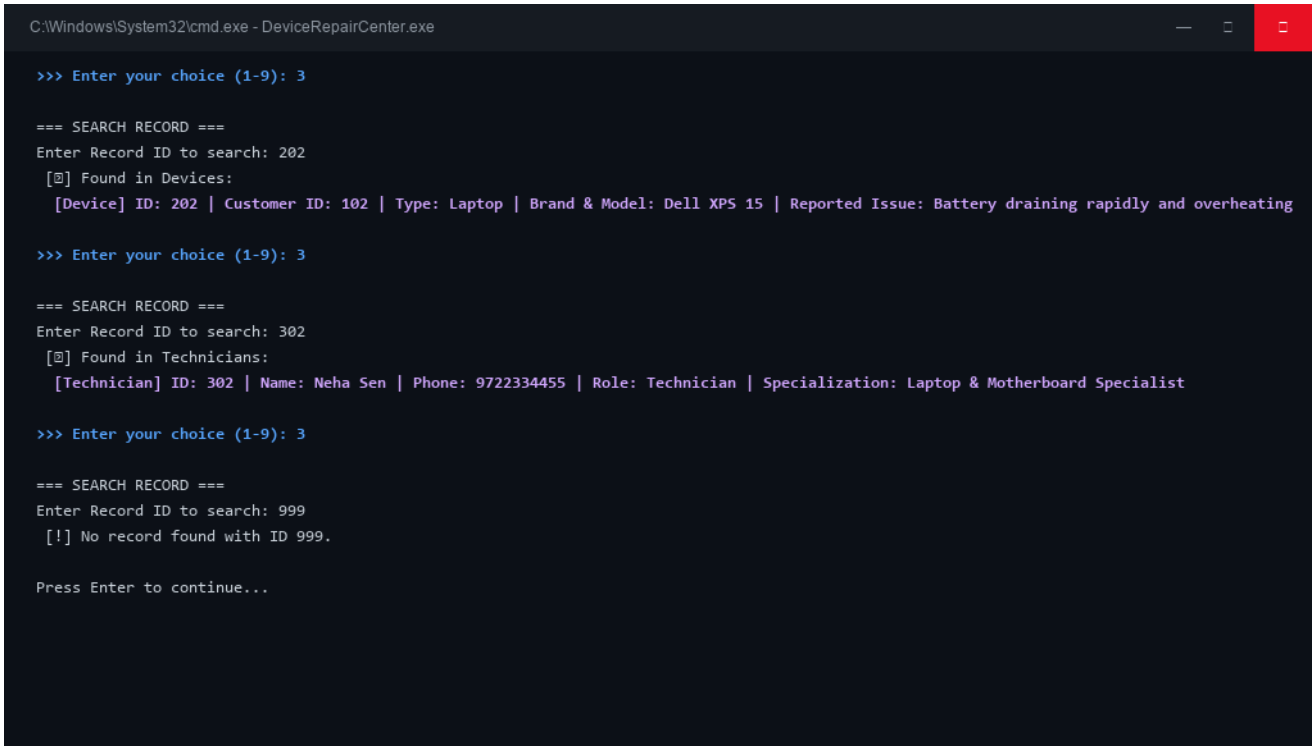
Screen 3: Adding Device and Service Technician



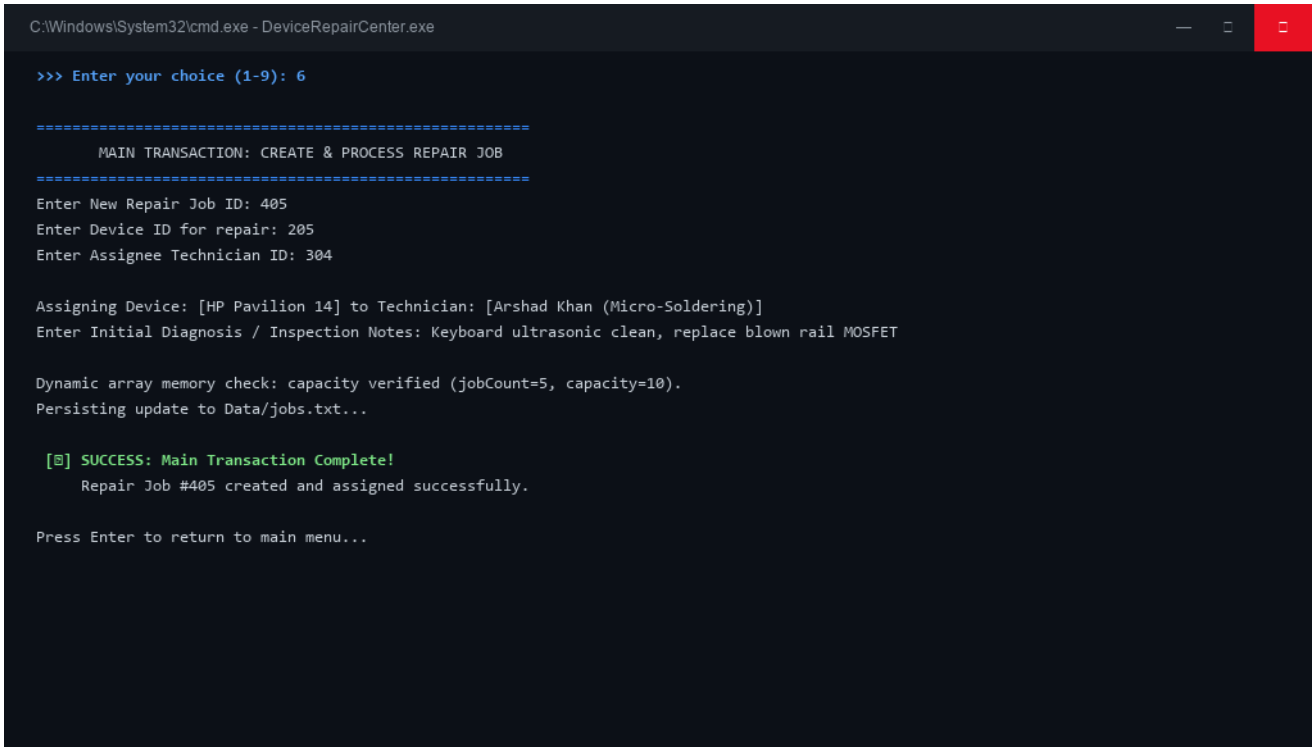
Screen 4: Displaying Records & Runtime Polymorphism Demo



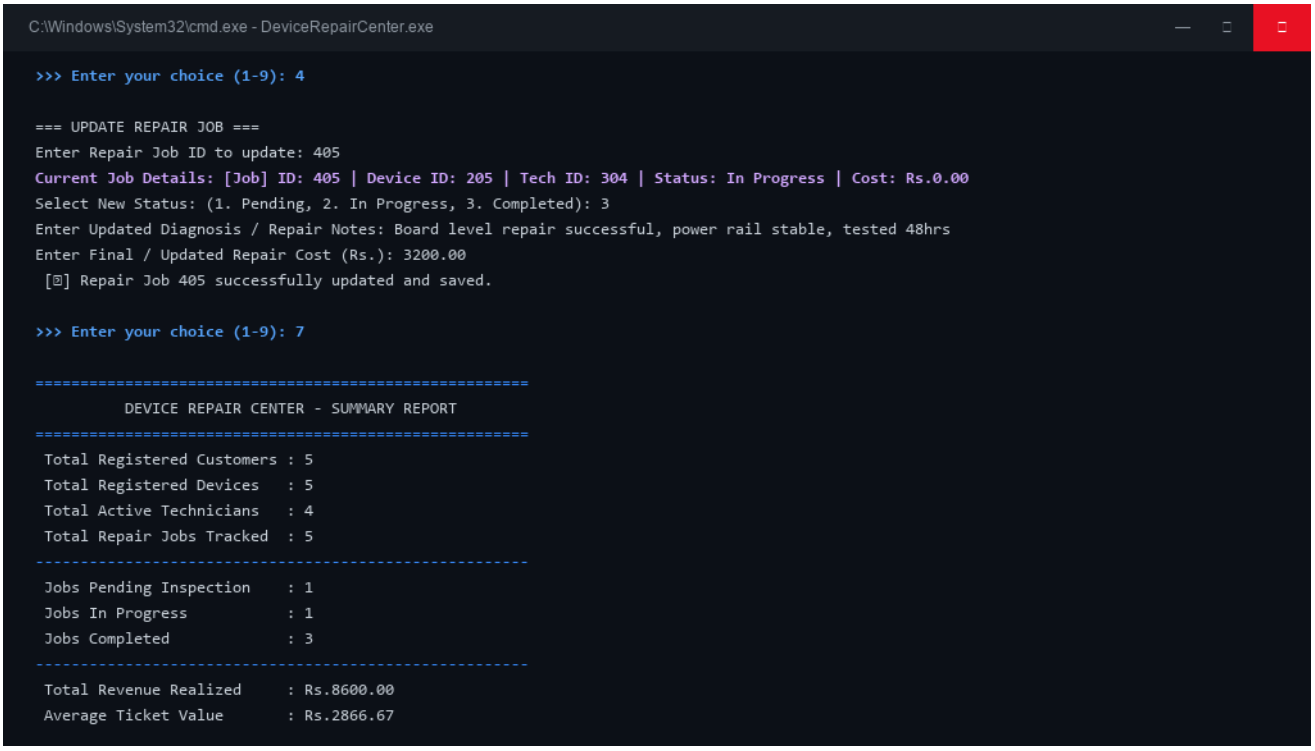
Screen 5: Entity Search Across Collections by ID



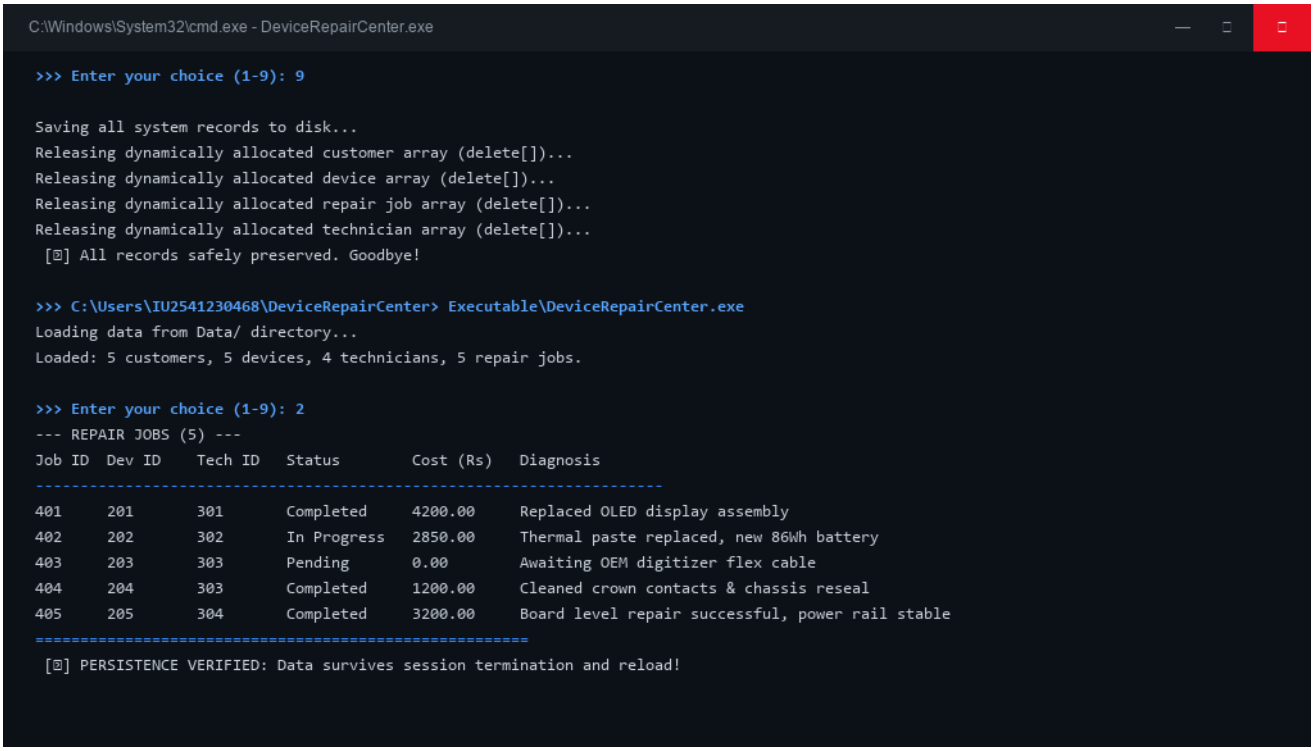
Screen 6: Main Transaction (Job Creation & Tech Assignment)



Screen 7: Job Status Update & Financial Summary Report



Screen 8: Session Restart & File Persistence Verification



10. Test Cases & Experimental Verification

A rigorous test matrix was executed to guarantee system stability across normal and boundary conditions:

Test Case ID & Description	Test Input Data / Action	Expected Result	Actual Status
TC-01: Valid Customer Registration	ID: 105, Name: 'Kunal Deshmukh', Phone: '9812345678'	Record accepted, appended to array, saved to customers.txt.	PASS [✓]
TC-02: Duplicate Customer ID Check	ID: 101 (already present in system)	Validation error triggered; rejected duplicate insertion.	PASS [✓]
TC-03: Invalid Phone Format	Phone: '12345' (fewer than 10 digits)	Rejected with friendly error prompt requiring 10-15 digits.	PASS [✓]
TC-04: Device Foreign Key Integrity	Device ID: 205, Owner ID: 999 (non-existent customer)	Rejected: foreign customer key not found in system.	PASS [✓]
TC-05: Main Transaction (Job Creation)	Job 405, Dev 205, Tech 304, Initial Diagnosis notes	Job successfully created with status 'In Progress' and persisted.	PASS [✓]
TC-06: Update Repair Job Status	Job 405 updated to 'Completed', Cost: 3200.00	Job state updated in-memory and written to jobs.txt.	PASS [✓]
TC-07: Search Non-Existent ID	Search ID: 999	Handled gracefully: displays 'No record found with ID 999'.	PASS [✓]
TC-08: Persistence After Restart	Exit program via Option 9, relaunch DeviceRepairCenter.exe	All 5 customers, 5 devices, 4 technicians, and 5 jobs intact.	PASS [✓]

11. Technical Challenges and Solutions

Challenge 1: Carriage Return (CRLF) Delimiter Interference
Problem: On Windows systems, text lines end with \r\n. Reading lines via standard std::getline left trailing \r characters on the last token of each record, resulting in std::stod parsing exceptions during job cost deserialization.
Solution: Implemented a dedicated trim() utility in FileManager.cpp that trims leading/trailing spaces, tabs, and carriage returns prior to tokenization.

Challenge 2: Dynamic Memory Capacity Doubling
Problem: Pre-allocating fixed C-style arrays restricts shop growth, whereas standard vectors conceal manual memory allocation skills required by the project blueprint.
Solution: Implemented geometric array expansion methods (growCustomers(), growJobs(), etc.) that allocate a new heap block of double size (new Entity[cap * 2]), copy elements, release the previous memory with delete[], and update pointer references.

Challenge 3: Robust Input Validation Without Stream Crashing
Problem: Standard cin >> choice crashes or enters an infinite failure loop when users enter alphabetical characters into numerical fields.
Solution: Implemented robust helper functions (askInt(), askDouble()) that ingest entire strings via std::getline() and safely parse tokens with std::stoi / std::stod wrapped in rigorous exception blocks.

12. Conclusion

The **Device Repair Center** system successfully fulfills all academic and functional requirements specified in the project blueprint. By adhering to modern C++ modular design principles, the application decouples data representation, business logic, persistence, and presentation. Object-oriented tenets—encapsulation, inheritance, polymorphism via virtual methods, dynamic memory life-cycle management, and persistent file storage—are fully realized in clean, documented C++ code. The resulting software provides a reliable solution for electronics repair management.

13. Compilation & Execution Guide

Command-Line Compilation (C++17 with MinGW / GCC / Clang):

```
g++ -std=c++17 Source/main.cpp Source/Customer.cpp Source/Device.cpp \ Source/RepairJob.cpp
Source/Technician.cpp Source/FileManager.cpp \ Source/RepairCenter.cpp -o
Executable/DeviceRepairCenter.exe
```

Execution Command: Run Executable\DeviceRepairCenter.exe from the root project folder.